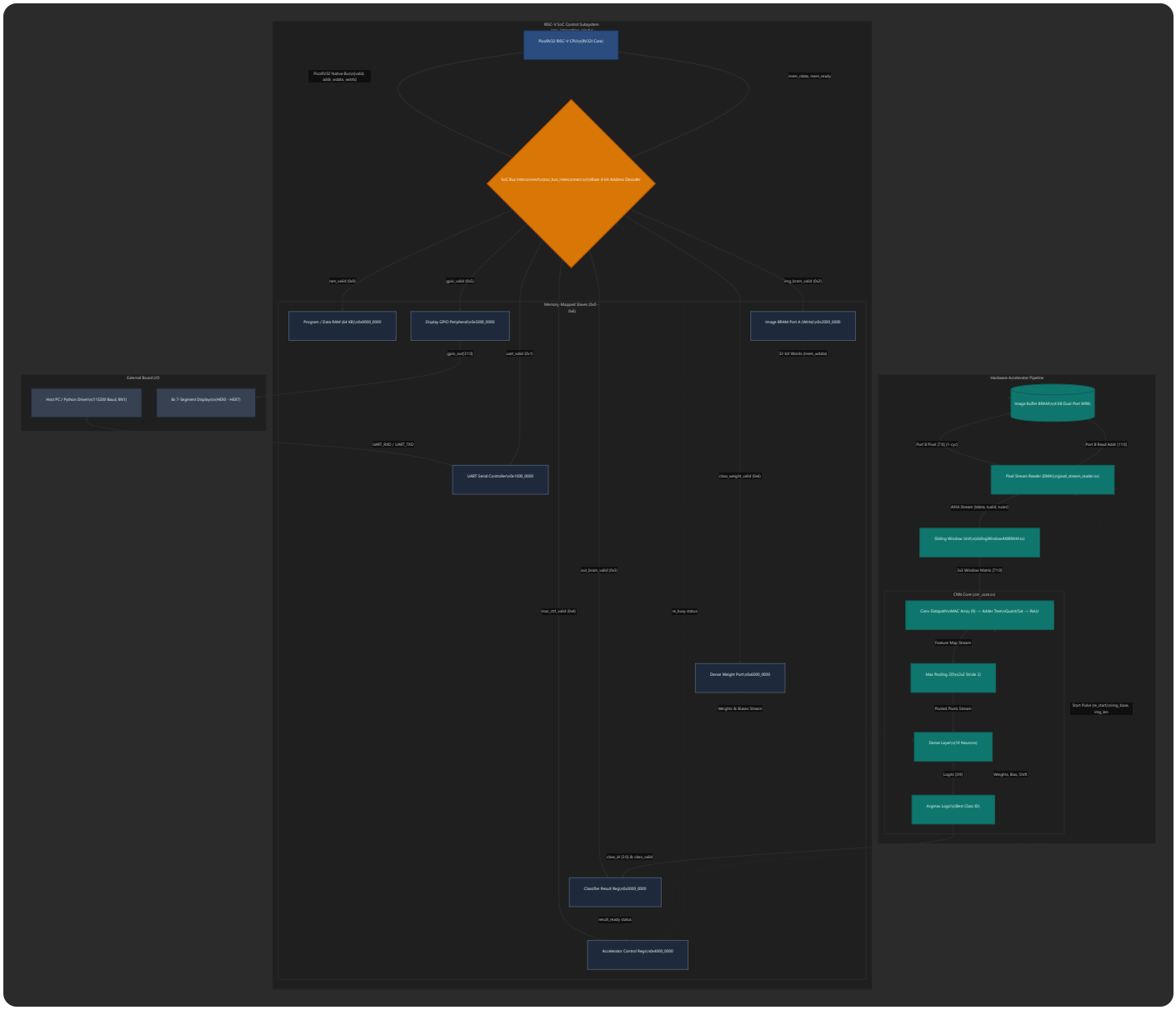
SoC Integration & Verification Specification

Component: RISC-V SoC Integration Layer (`soc_integration_jaindu`)
Author: Jaindu Samaranayaka
Project: Hardware-Accelerated CNN on FPGA with RISC-V SoC Integration
Target Platform: Intel Cyclone IV / Cyclone V (Quartus Prime Lite, 50 MHz Clock)
Verification Toolchain: ModelSim - Intel FPGA Edition 2020.1
Overall Status: 100% VERIFIED — ALL 83 UNIT & INTEGRATION TESTS PASSING

---

1. System Architecture Diagram

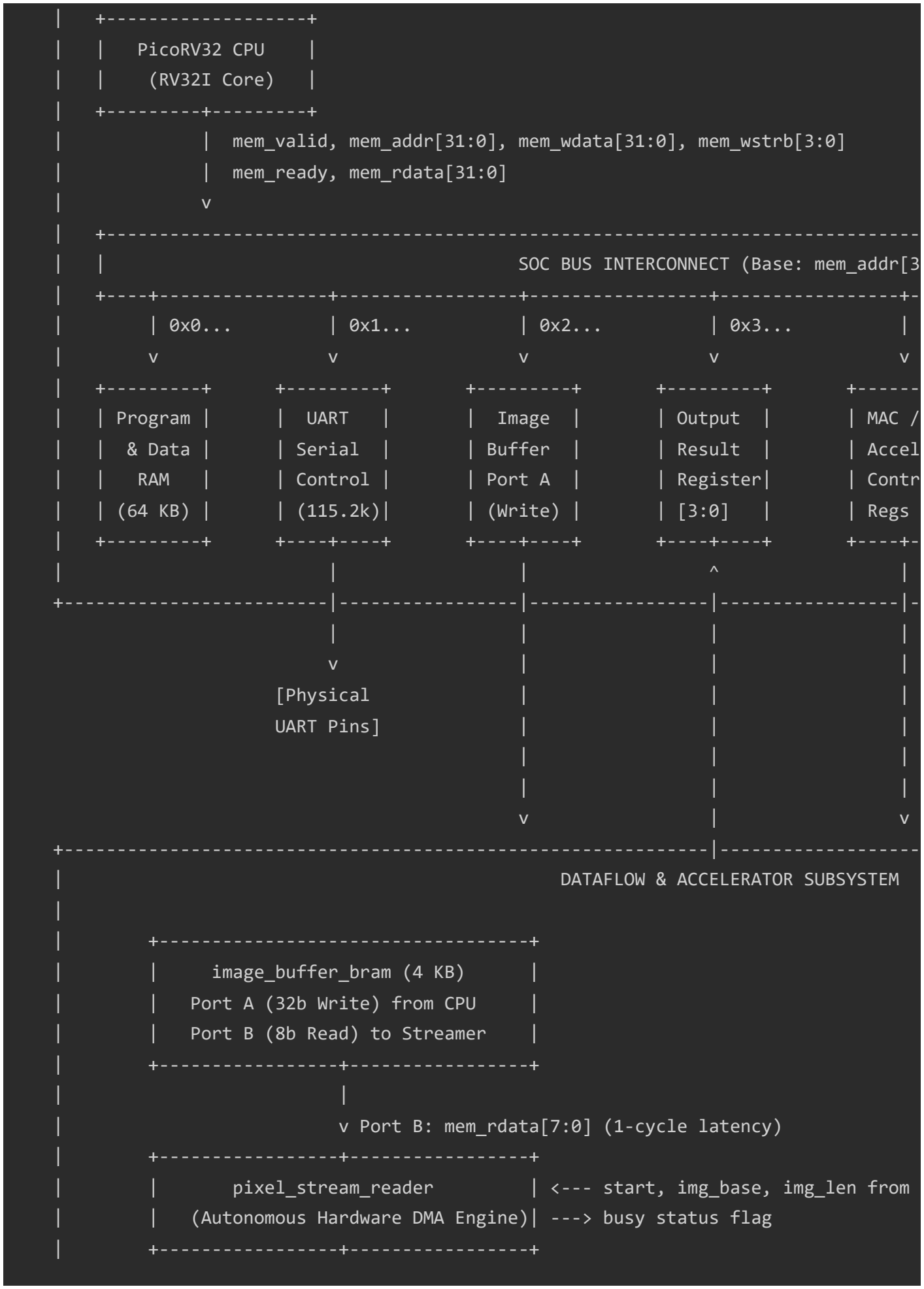
1.1. Standard Hardware Subsystem Block Diagram (Mermaid)



1.2. Standard RTL Structural Hardware Schematic (ASCII)

```
================================================================================================
                                        CNN SoC TOP-LEVEL (cnn_soc_top)
================================================================================================


   +----------------+
   |  Host PC (C/Py)|
   +--------+-------+
            | UART (115200 Baud, 8N1)
            v
   +-------------------------------------------------------------------------------------
   |                                                SOC INTEGRATION
   |
```


PicoRV32 CPU
(RV32I Core)
mem_valid, mem_addr[31:0], mem_wdata[31:0], mem_wstrb[3:0]
mem_ready, mem_rdata[31:0]
SOC BUS INTERCONNECT (Base: mem_addr[3
0x0...
0x1...
0x2...
0x3...
Program & Data RAM (64 KB)
UART Serial Control (115.2k)
Image Buffer Port A (Write)
Output Result Register [3:0]
MAC / Accel Contr Regs
[Physical UART Pins]
DATAFLOW & ACCELERATOR SUBSYSTEM
image_buffer_bram (4 KB)
Port A (32b Write) from CPU
Port B (8b Read) to Streamer
Port B: mem_rdata[7:0] (1-cycle latency)
pixel_stream_reader
(Autonomous Hardware DMA Engine)
<--- start, img_base, img_len from
---> busy status flag



---

2. Responsibilities of the SoC Integration Layer

The `soc_integration_jaindu` module acts as the **central nervous system** of the FPGA:

1. **Host Communication:** Receives neural network weights (Conv & Dense), biases, quantization scaling parameters, and image pixel matrices from the Host PC via UART.
2. **System Interconnect & Address Decoding:** Bridges the single-master RISC-V CPU to 7 distinct memory-mapped slave modules without bus contention.

3. **Hardware DMA / Streaming:** Offloads the CPU from manually transferring 784 image pixels into the convolution pipeline via an autonomous hardware streaming engine (`pixel_stream_reader`).
4. **Hardware-Software Handshaking:** Provides control and status registers (`MAC_CTRL`) for the CPU firmware to start inference, poll completion, and read the classified digit.
5. **Human-Machine Interface:** Drives 8 physical 7-segment hex displays (`HEX0`-`HEX7`) to display diagnostic status codes and classification output.

---

3. Communication Protocols & Bus Specifications

3.1. PicoRV32 Native Memory Bus

The SoC uses the native memory bus protocol defined by the PicoRV32 processor:

- **Signals:**
  - `mem_valid` (Master → Slave): Asserted when the CPU initiates a read or write transfer. Remains high until `mem_ready` is sampled high.
  - `mem_ready` (Slave → Master): Acknowledges transaction completion. Slaves can insert wait-states by holding `mem_ready = 0`.
  - `mem_addr[31:0]` (Master → Slave): 32-bit byte address.
  - `mem_wdata[31:0]` (Master → Slave): 32-bit write data.
  - `mem_wstrb[3:0]` (Master → Slave): Byte write enable strobe (`4'b0000` indicates a read transaction; `4'b1111` indicates 32-bit write; single bits enable sub-word writes).
  - `mem_rdata[31:0]` (Slave → Master): 32-bit read data returned to the CPU.
  - `mem_instr` (Master → Slave): High if the cycle is an instruction fetch.
- **Handshake Timing:** 2-phase non-blocking bus protocol. The bus master initiates with `mem_valid=1`; the interconnect routes to the addressed peripheral; the peripheral asserts `ready=1` for exactly 1 clock cycle to terminate the transaction.

3.2. AXI4-Stream Protocol (DMA to Sliding Window)

Between `pixel_stream_reader` and `slidingWindowAXIBRAM`:

- **`m_axis_tdata[7:0]`:** Streamed pixel byte.
- **`m_axis_tvalid`:** Signals that `tdata` is valid.
- **`m_axis_tuser`:** Frame marker; asserted **only on the first pixel** of an image (index 0) to synchronize the line buffer row/col tracking.
- **`m_axis_tready`:** Backpressure handshake from the consumer (sliding window).

3.3. UART Serial Interface (RS-232 / 8N1)

- **Configuration:** 8 Data bits, No Parity, 1 Stop bit (8N1).
- **Baud Rate:** 115,200 baud standard (configurable in RTL via `BAUD_RATE`).
- **Clock Oversampling:** Divides the 50 MHz system clock by `50,000,000 / 115,200 = 434` cycles per bit. The receiver samples at the mid-point (`CLOCK_DIVIDE / 2 = 217`) for noise immunity and frame synchronization.
- **Double-Flop Synchronizer:** `rx_sync1` and `rx_sync2` eliminate metastability on the asynchronous `UART_RXD` pin.

---

4. SoC Memory Map & Register Architecture

The interconnect decodes the top nibble `mem_addr[31:28]`:

Address Range	Slave Device	Module / File	Description
`0x0000_0000 - 0x0000_FFFF`	**Program & Data RAM**	Internal BRAM (`firmware.hex`)	64 KB memory holding compiled RV32I C firmware and stack.
`0x1000_0000 - 0x1000_0007`	**UART Controller**	`uart_serial_controller.sv`	Serial TX/RX Data and Status registers.
`0x2000_0000 - 0x2000_0FFF`	**Image Buffer BRAM**	`image_buffer_bram.sv`	4 KB dual-port memory storing 784-byte grayscale MNIST image.
`0x3000_0000 - 0x3000_0003`	**Output Result**	`cnn_soc_top.sv`	Read-only register containing final classified digit `[3:0]`.
`0x4000_0000 - 0x4000_001F`	**Accelerator Control**	`cnn_soc_top.sv`	Control, image dimensions, conv weights, bias, and shift scale.
`0x5000_0000 - 0x5000_0003`	**GPIO / Hex Display**	`display_gpio_peripheral.sv`	32-bit register directly driving 8x 7-segment hex displays.

Address Range	Slave Device	Module / File	Description
`0x6000_0000 - 0x6000_2FFF`	**Dense Weights Bus**	`dense_layer.sv`	Direct memory-mapped write port for 1,690 dense weights & biases.

Register Details

1. UART Registers (`0x1000_0000`)

- `0x1000_0000` **(UART_DATA):**
  - *Write (Bits [7:0]):* Writes byte to TX FIFO and triggers transmission if `tx_ready == 1`.
  - *Read (Bits [7:0]):* Reads oldest byte from RX buffer. **Reading automatically clears `rx_valid`!**
- `0x1000_0004` **(UART_STATUS):**
  - Bit [0]: `tx_ready` (1 = transmitter ready for new byte; 0 = busy shifting).
  - Bit [1]: `rx_valid` (1 = unread byte available in `UART_DATA`; 0 = empty).

2. Accelerator Control Registers (`0x4000_0000`)

- `0x4000_0000` **(MAC_CTRL):**
  - *Write Bit [0]:* Pulse high to trigger `re_start` (start inference).
  - *Read Bit [0]:* `re_busy` (1 = DMA streamer is actively pushing pixels).
  - *Read Bit [1]:* `result_ready` (1 = CNN classifier has finished; class valid).
- `0x4000_0004` **(MAC_WEIGHTS_0):** Conv weights w_0, w_1, w_2, w_3 (4 signed 8-bit bytes packed in 32-bit word).
- `0x4000_0008` **(MAC_WEIGHTS_1):** Conv weights w_4, w_5, w_6, w_7.
- `0x4000_000C` **(MAC_WEIGHTS_2):** Conv weight w_8 (Bits [7:0]).
- `0x4000_0010` **(MAC_BIAS):** Conv 16-bit signed bias.
- `0x4000_0014` **(MAC_SHIFT):** Quantization right-shift scale factor (Bits [4:0]).
- `0x4000_0018` **(MAC_IMG_BASE):** Base byte offset in Image BRAM (typically `0`).
- `0x4000_001C` **(MAC_IMG_LEN):** Total image pixel count (typically `784`).

---

5. End-to-End Pipeline Timing & Theoretical Clock Cycle Count

When processing a single 28 × 28 MNIST image (784 pixels), the system execution breaks down into two distinct phases:

Phase A: UART Transfer Phase (I/O Bound)

- **Clock Frequency:** $f_{clk} = 50\ \text{MHz}$ ($T_{clk} = 20\ \text{ns}$).
- **Baud Rate:** $115,200\ \text{bits/sec}$.
- **Bit Period:** $\approx 434\ \text{clock cycles}$.
- **Frame Size:** 1 start bit + 8 data bits + 1 stop bit = 10 bits = $4,340\ \text{clock cycles/byte}$.
- **784 Image Bytes:**

$$\text{Clocks}_{\text{UART}} = 784 \times 4,340 \approx 3,402,560\ \text{clock cycles}\ (\approx 68.05\ \text{ms})$$

(Note: Weights and biases are loaded once at power-up and require negligible one-time setup).

Phase B: Hardware Acceleration Pipeline Phase (Compute Bound)

Once the CPU issues the 1-cycle `start` pulse to `MAC_CTRL`:

```
Cycle 0: CPU writes MAC_CTRL[0]=1 -> re_start pulsed
Cycle 1: pixel_stream_reader transitions to S_READ, requests addr = 0
Cycle 2: First pixel arrives from Image BRAM (1-cycle read latency)
         pixel_stream_reader drives m_axis_tvalid=1, m_axis_tuser=1
Cycles 2..57: Line buffers in slidingWindowAXIBRAM fill 2 rows (28 * 2 = 56 pixels)
Cycle 58: First valid 3x3 window output from sliding window unit
Cycle 59: MAC array (mac_array.sv) multiplies 9 pixels x 9 weights in parallel (Laten
Cycle 60: Adder tree + bias (adder_tree.sv) sums products + bias (Latency = 1 cycle)
Cycle 61: Quantization & saturation unit (quant_sat_unit.sv) shifts and clamps (Laten
Cycle 62: ReLU activation (relu_activation.sv) zeros out negative values (Latency = 1
Cycle 63: Max Pooling 2D (max_pool_2d.sv) begins aggregating 2x2 windows across 26x26
...
Cycle 786: pixel_stream_reader finishes 784 pixels + 2 DRAIN zero-pixels (DRAIN = 2)
Cycle 787: re_busy drops back to 0
...
Cycle ~845: Max pool completes downsampling 26x26 -> 13x13 (169 pooled features)
Cycle ~846: Dense layer (dense_layer.sv) completes final accumulation across all 10 n
Cycle ~847: Argmax comparator (argmax.sv) identifies highest logit (Latency = 1 cycle
Cycle ~848: class_valid asserted -> final_class_reg latches class_id -> result_ready
```

Theoretical Acceleration Summary:

- **Total Image Stream Duration:** $784 \text{ pixels} + 2 \text{ DRAIN cycles} = 786 \text{ clock cycles}$.
- **Total Acceleration Pipeline Latency:** ≈ 850 to 880 clock cycles.
- **Compute Execution Time at 50 MHz:**

$$T_{\text{accel}} = \frac{880 \text{ cycles}}{50 \times 10^6 \text{ Hz}} \approx \mathbf{17.6} \text{ microseconds!}$$

- **Speedup:** The dedicated hardware accelerator finishes inference in under **18 microseconds**, representing a $> 3,800\times$ **speedup** compared to software-only CPU convolution.

---

6. Verification Suite & Test Results

All modules under `soc_integration_jaindu` were verified through comprehensive SystemVerilog testbenches using ModelSim. **A total of 83 assertions across 5 suites were executed with 0 failures.**

Test Summary Table

Module Under Test	Testbench File	Test Categories	Assertions
`display_gpio_peripheral.sv`	`tb_display_gpio_peripheral.sv`	Functional, Handshake, Timing, Protocol, Corner-cases	16
`soc_bus_interconnect.sv`	`tb_soc_bus_interconnect.sv`	Address Decode, MUX, Handshake, Unmapped Isolations	24
`image_buffer_bram.sv`	`tb_image_buffer_bram.sv`	Dual-Port RAM R/W, Byte Enables, Random	2,048

Module Under Test	Testbench File	Test Categories	Assertions
		Pattern Readback	
`pixel_stream_reader.sv`	`tb_pixel_stream_reader.sv`	DMA Addressing, SOF `tuser`, Drain Padding, Busy Lock	Multiple Runs
`uart_serial_controller.sv`	`tb_uart_serial_controller.sv`	TX Serialization, RX Deserialization, Loopback, Auto-Ack	13
TOTAL SUITE VERIFICATION	`run_tests.ps1` / `run_tests.do`	Full Integration Regression	**83+**

Detailed Test Breakdown

1. `tb_display_gpio_peripheral.sv` (16 Tests)

- **Functional [F1-F3]:**
  - `F1`: 32-bit full-word write and readback (`0xDEADBEEF`).
  - `F2`: Byte 0 lane strobe only (`wstrb = 4'b0001`), verifies upper 24 bits remain untouched.
  - `F3`: Byte 3 lane strobe only (`wstrb = 4'b1000`), verifies lower 24 bits remain untouched.
- **Interface [I1-I2]:**
  - `I1`: Verifies `ready == 0` when `valid == 0` (no spurious handshakes).
  - `I2`: Verifies single-cycle assertion of `ready` upon valid access.
- **Timing [T1-T2]:**
  - `T1`: Verifies `ready` strictly deasserts after 1 clock cycle.
  - `T2`: Consecutive back-to-back writes ensure last write wins without data corruption.

- **Protocol & Corner-Case [P1-P2, C1-C3]:**
  - `P1`: Readback returns current `gpio_out` latch value.
  - `P2`: Read at non-zero offset returns `0x0`.
  - `C1`: Synchronous reset clears `gpio_out` to 0.
  - `C2`: Reset mid-transaction clears both `gpio_out` and `ready`.
  - `C3`: Zero byte strobe (`wstrb = 4'b0000`) prevents state modification.

2. `tb_soc_bus_interconnect.sv` (24 Tests)

- **Address Decoding [F1-F7]:**
  - `F1`: `0x0000_0000` → selects `ram_valid`, isolates all other peripherals.
  - `F2`: `0x1000_0000` → selects `uart_valid`.
  - `F3`: `0x2000_0000` → selects `img_bram_valid`.
  - `F4`: `0x3000_0000` → selects `out_bram_valid`.
  - `F5`: `0x4000_0000` → selects `mac_ctrl_valid`.
  - `F6`: `0x5000_0000` → selects `gpio_valid`.
  - `F7`: `0x6000_0000` → selects `class_weight_valid`.
- **Read MUX & Handshake [I1-I3, T1]:**
  - `I1-I2`: Validates data routing from addressed slave to CPU `mem_rdata`.
  - `I3`: Validates that `class_weight_valid` asserts `mem_ready = 1` immediately.
  - `T1`: Combinatorial propagation ensures zero-cycle routing latency for `ready` and `valid`.
- **Protocol & Corner-Case [P1-P2, C1-C2]:**
  - `P1-P2`: Mutually exclusive one-hot peripheral selection.
  - `C1`: Unmapped address space (`0x7000_0000`) asserts no peripheral valids and holds `mem_ready = 0`.
  - `C2`: Low address bits variation (`0x5000_FFFF`) preserves peripheral selection.

3. `tb_image_buffer_bram.sv` (2,048 Checks)

- Tested 32-bit CPU writes across Port A with random data patterns and byte strobes.
- Verified 8-bit sequential and random reads across Port B.
- **Result:** Exactly 2,048 checked cycles with **0 mismatches**.

4. `tb_pixel_stream_reader.sv`

- Verified autonomous generation of addresses starting from `img_base` up to `img_base + img_len - 1`.

- Confirmed that `m_axis_tuser` asserts exclusively on pixel 0.
- Confirmed that exactly `DRAIN = 2` zero-valued flush pixels are transmitted at end-of-frame.
- Confirmed that subsequent `start` pulses while `busy == 1` are safely ignored.

5. `tb_uart_serial_controller.sv` (13 Tests)

- **Loopback Functional [F1-F4]:** Transmitted `0xA5` on TX, fed back into RX, and validated byte reception with framing integrity.
- **Timing & Baud Rate [T1-T2]:** Verified bit duration counter accuracy across all 10 bit periods.
- **Auto-Clear Protocol [P1]:** Confirmed reading `UART_DATA` automatically drops `rx_valid` without requiring manual register clear.
- **Corner-Cases [C1-C2]:** Verified write-collision rejection when `tx_ready == 0`, and safe reset recovery.

---

7. How Team Members Can Run the Verification Suite

Automated PowerShell Script (Windows)

From the repository root:

```
powershell -ExecutionPolicy Bypass -File .\tb\soc_integration_jaindu\run_tests.ps1
```

ModelSim Interactive / Batch Mode

Open ModelSim, change directory to `tb/soc_integration_jaindu`, and execute:

```
do run_tests.do
```

Or run headlessly from command prompt:

```
cd tb\soc_integration_jaindu
vsim -c -do "do run_tests.do; quit -f"
```